qbo_social — Architecture & Documentation

> **Dernière mise à jour :** avril 2026
> Évolutions majeures : `DiagnosticsInspector` (watchdog nœuds), `NODE_PROFILES`, dashboard temps réel, extracteurs réels depuis sources C++/Python.

Vue d'ensemble

`qbo_social` est le **Social Behavior Engine (SBE)** du robot Qbo.
Il orchestre toute la logique sociale : détection de visages, reconnaissance de personnes, diagnostics matériels, et gestion des modes système (batterie faible, charge…).

Il est organisé en **pipeline de nœuds ROS2 indépendants** qui communiquent via des topics.

Architecture générale

```
Capteurs & drivers externes
  │
  ├── /qbo_face_following/status        → FaceAdapter
  ├── /qbo_face_recognition/result      → FaceAdapter
  └── /diagnostics ──────────────────→  DiagnosticsAdapter  (events WARNING/ERROR + robot_state complet)
                                      └→ DiagnosticsInspector (watchdog présence nœuds)
                                      └→ SystemModeManager    (extracteurs → robot_state → règles)
                                                 │
                                           SocialEventAdapter
                                           (event_adapter.py)
                                                 │
                                        /qbo_social/events
                                                 │
                                 ┌───────────────┴──────────────────┐
                                 ↓                                  ↓
                        SocialWorldModel                   SocialBehaviorEngine
                        (world_model.py)                  (behavior_engine.py)
                                 │                                  │
                   /qbo_social/world_state               /qbo_social/intent_raw
                                 │                                  │
                                 └──────────→ (lecture) ←───────────┘
                                                 │
                                        SystemModeManager
                                      (system_mode_manager.py)
                                  (profils + filtre + dégradation)
                                                 │
                                        /qbo_social/intent
                                                 │
                                        SocialActionExecutor
                                        (action_executor.py)
                                   (LED, face follower, START_PROFILE…)
```

Les nœuds

1. `event_adapter` — Normalisation des événements

Fichier : `qbo_social/event_adapter.py`
Topic publié : `/qbo_social/events` (`SocialEvent`)

Point d'entrée du pipeline. Héberge deux **adapters** qui convertissent les messages ROS2 bruts en événements sociaux normalisés :

Adapter	Source	Événements produits
`FaceAdapter`	`/qbo_face_following/status`, `/qbo_face_recognition/result`	`FACE_APPEARED`, `FACE_STABLE`, `FACE_LOST`, `PERSON_RECOGNIZED`
`DiagnosticsAdapter`	`/diagnostics`	`DIAGNOSTIC` (WARNING ou ERROR) + stocke tout dans `robot_state[hw][category]`
`DiagnosticsInspector`	`/diagnostics`	`NODE_MISSING`, `NODE_RECOVERED` + `robot_state["nodes_present"]`

Paramètres (`qbo_social_params.yaml`) :

```yaml
qbo_social_event_adapter:
  ros__parameters:
    face_stable_time: 3.0     # secondes avant de considérer un visage stable
    face_lost_timeout: 2.0    # secondes avant de considérer un visage perdu
    min_confidence: 0.6       # seuil minimum pour accepter une reconnaissance
```

---

2. `world_model` — État du monde

Fichier : `qbo_social/world_model.py`
Souscrit : `/qbo_social/events`
Publie : `/qbo_social/world_state` (`WorldState`) — toutes les 500ms + à chaque événement

Maintient une **image cohérente de la situation** à l'instant t. C'est l'équivalent de `self.robot_state` de l'ancien système AIML.

Champs clés du `WorldState` :

Champ	Type	Description
`face_present`	bool	Un visage est visible
`face_stable`	bool	Visage stabilisé depuis `face_stable_time`
`conversation_active`	bool	Conversation en cours
`focus_person_id`	string	ID de la personne suivie
`focus_person_name`	string	Nom de la personne suivie
`engagement_level`	float	0.0 → 1.0
`mode`	string	`IDLE`, `SOCIAL_ACTIVE`, `ENGAGED`
`health_state`	string	`NORMAL`, `LOW_BATTERY`, `CHARGING`, `CRITICAL`

Mise à jour de `health_state` : le `WorldModel` écoute les événements `SYSTEM_MODE_CHANGED` publiés par le `SystemModeManager`.

---

3. `behavior_engine` — Décisions comportementales

Fichier : `qbo_social/behavior_engine.py`
Souscrit : `/qbo_social/events`, `/qbo_social/world_state`
Publie : `/qbo_social/intent_raw` (`BehaviorIntent`), `/qbo_social/decision_trace`

Prend les **décisions** : que doit faire le robot face à un événement donné ? Il vérifie le `WorldState` avant d'agir (cooldowns, conditions sociales...).

Machine à états du face tracking :

```
[IDLE] ─────── FACE_APPEARED ──────→ [TRACKING_FULL]
                                          │
                                          │ face_stable + 5s
                                          ▼
[IDLE] ←── search_timeout ─── [TRACKING_HEAD_ONLY]
  ▲              30s                      │
  │                                       │ FACE_LOST
  │                                       ▼
  └──────────────────────────────  [SEARCHING]
                                          │
                                          │ FACE_APPEARED
                                          └──────────────→ [TRACKING_FULL]
```

Intents produits :

Intent	Déclencheur
`TRACK_FACE_FULL`	`FACE_APPEARED`
`TRACK_FACE_HEAD_ONLY`	Visage stable depuis `stable_head_stop_delay`
`START_PERSON_SEARCH`	`FACE_LOST`
`STOP_PERSON_SEARCH`	Timeout de recherche dépassé
`GREET_PERSON`	`PERSON_RECOGNIZED` (si face_stable + cooldown OK)
`SPEAK`	Diagnostic WARNING (si IDLE) ou ERROR

Paramètres (`qbo_social_params.yaml`) :

```yaml
qbo_social_behavior_engine:
  ros__parameters:
    greet_cooldown: 10.0         # secondes entre deux salutations
    stable_head_stop_delay: 5.0  # secondes avant d'arrêter la tête quand visage stable
    search_timeout: 30.0         # secondes de recherche max après perte du visage
    diag_cooldown: 5.0           # secondes entre deux annonces diagnostics
```

---

4. `system_mode_manager` — Filtre d'intents, mode système & profils

Fichier : `qbo_social/system_mode_manager.py`
Souscrit : `/diagnostics`, `/qbo_social/intent_raw`, `/qbo_social/events`
Publie : `/qbo_social/intent` (filtrés/dégradés), `/qbo_social/events` (`SYSTEM_MODE_CHANGED`)
Paramètre : `active_profile` (modifiable à chaud via `ros2 param set`)

Flux :

```
BehaviorEngine → intent_raw → [filtre règles] → intent → ActionExecutor
                 /diagnostics ──→ EXTRACTORS ──→ robot_state ──→ SYSTEM_RULES ──→ filtre
                 /events       ──→ NODE_MISSING/RECOVERED ──→ nodes_present
                                                                  │
                                 NODE_PROFILES (profil actif) ──────────→ watchdog timer (5s)
                                                                  │
                                              START_PROFILE intent ←──┘ (si nœud manquant)
                                                                  │
                                                        SYSTEM_MODE_CHANGED → events
```

Profils de nœuds (`NODE_PROFILES`)

Un profil liste les `hardware_id` **requis** pour que le robot soit considéré opérationnel. Si un nœud requis est absent (détecté par `DiagnosticsInspector`), un intent `START_PROFILE` est publié vers l'`ActionExecutor` (qui lancera `ros2 launch qbo_bringup profile_X.launch.py` — étape 2).

Profil	Nœuds requis	Inclut
`MINIMAL`	`System`, `Qboard_1`, `Qboard_3`, `orin-nx-16g`	—
`VISION`	`qbo_vision`, `qbo_dynamixel`	MINIMAL
`NAVIGATION`	*(à compléter)*	MINIMAL
`FULL`	—	VISION + NAVIGATION

Changer le profil en live :

```
ros2 param set /qbo_system_mode_manager active_profile VISION
```

Carte des `hardware_id` réels (vérifiés depuis les sources)

`hardware_id`	`name` (catégorie)	Fichier source	Clés de valeurs utiles
`System`	`Arduqbo Status`	`qbo_arduqbo.cpp`	contrôleurs activés
`Qboard_1`	`Base Status`, `Sensors Status`	`base_controller.cpp`, `sensor_controller.cpp`	état moteurs, capteurs
`Qboard_3`	`Battery Status`	`battery_controller.cpp`	`Voltage`, `Charge Mode`, `Estimated Runtime`
`Qboard_4`	`IMU Status`	`imu_controller.cpp`	`Gyroscope Present`, `IMU calibrated`
`Qboard_5`	`Nose Status`, `Mouth Status`	`nose_controller.cpp`, `mouth_controller.cpp`	`color_name`
`LCD`	`LCD Status`	`lcd_controller.cpp`	`LCD Present`
`qbo_dynamixel`	`head_pan_joint`, `head_tilt_joint`	`dynamixel_controller.cpp`	position, temp, voltage
`orin-nx-16g`	`A608 Power/Temp/Fan/Network`	`hardwareOrinA608.py`	`Voltage`, `CPU °C`, `RAM %`, `VDD_IN W`
`qbo_vision`	`face_tracker`	`face_tracker.cpp`	`tracking_state`, `video_stream_ok`

Extracteurs actifs (`DIAGNOSTIC_EXTRACTORS`)

Clé `robot_state`	`name_pattern`	`value_key`	Source
`battery_voltage`	`Battery Status`	`Voltage`	`Qboard_3`
`charging`	`Battery Status`	`Charge Mode`	`Qboard_3` (1 ou 2 = True)
`battery_runtime_min`	`Battery Status`	`Estimated Runtime`	`Qboard_3`
`cpu_temp_c`	`A608 Temp`	`CPU °C`	`orin-nx-16g`
`gpu_temp_c`	`A608 Temp`	`GPU °C`	`orin-nx-16g`

Clé `robot_state`	`name_pattern`	`value_key`	Source
`ram_percent`	`A608 Fan`	`RAM %`	`orin-nx-16g`
`cpu_percent`	`A608 Fan`	`CPU %`	`orin-nx-16g`
`total_power_w`	`A608 Power`	`VDD_IN W`	`orin-nx-16g`
`nodes_present`	*(DiagnosticsInspector)*	*(watchdog timeout)*	tous les nœuds

Règles actives (`SYSTEM_RULES`)

ID	Condition	Mode	Intents bloqués	Dégradations	Priorité
`VISION_MISSING`	`nodes_present["qbo_vision"] is False`	`DEGRADED_VISION`	TRACK_FACE_FULL, HEAD_ONLY, START_PERSON_SEARCH	—	60
`CRITICAL_BATTERY`	`battery_voltage < 11.5 V`	`CRITICAL`	GREET, DANCE, TRACK_FACE_*, MOVE_BASE, START_PERSON_SEARCH	—	100
`LOW_BATTERY`	`battery_voltage < 12.2 V`	`LOW_BATTERY`	DANCE, START_PERSON_SEARCH	TRACK_FACE_FULL → HEAD_ONLY	50
`CHARGING`	`charging == True`	`CHARGING`	MOVE_BASE	—	30
`CPU_OVERLOAD`	`cpu_temp_c >= 70.0 °C`	`THERMAL_LIMIT`	DANCE	TRACK_FACE_FULL → HEAD_ONLY	40

Ajouter une règle — méthode

Étape 1 : Si la donnée n'est pas encore dans `robot_state`, ajouter un extracteur dans `DIAGNOSTIC_EXTRACTORS` :

```
{
    "name_pattern": r"Battery Status",  # regex sur status.name
    "value_key": "Voltage",             # clé dans status.values[]
    "target": "battery_voltage",        # → robot_state["battery_voltage"]
    "transform": lambda v: float(v.strip()),
    "fallback": None,
},
```

Étape 2 : Ajouter la règle dans `SYSTEM_RULES` :

```
{
    "id": "MON_ID",
    "condition": lambda s: s.get("battery_voltage", 13.2) < 11.5,
    "mode": "CRITICAL",
    "block":   ["DANCE", "MOVE_BASE"],
    "degrade": {"TRACK_FACE_FULL": "TRACK_FACE_HEAD_ONLY"},
    "priority": 100,
},
```

Étape 3 : C'est tout. `WorldState.health_state` se met à jour automatiquement.

---

5. `action_executor` — Exécution

Fichier : `qbo_social/action_executor.py`
Souscrit : `/qbo_social/intent`
Publie : `/qbo_arduqbo/nose_ctrl/cmd_nose`, `/qbo_social/events`
Service : `/qbo_face_following/set_follower_status`

Traduit les intents en actions concrètes.

Intent	Action	Étape
`TRACK_FACE_FULL`	service face follower (head + base)	☑ actif
`TRACK_FACE_HEAD_ONLY`	service face follower (head seulement)	☑ actif
`GREET_PERSON`	LED verte + log	☑ actif
`START_PROFILE`	`ros2 launch qbo_bringup profile_X.launch.py`	⏳ étape 2
`STOP_NODE`	arrêt d'un nœud spécifique	⏳ étape 2

Topics résumés

Topic	Type	Producteur	Consommateurs
`/qbo_social/events`	`SocialEvent`	`event_adapter`, `system_mode_manager`	`world_model`, `behavior_engine`
`/qbo_social/world_state`	`WorldState`	`world_model`	`behavior_engine`
`/qbo_social/intent_raw`	`BehaviorIntent`	`behavior_engine`	`system_mode_manager`
`/qbo_social/intent`	`BehaviorIntent`	`system_mode_manager`	`action_executor`
`/qbo_social/decision_trace`	`DecisionTrace`	`behavior_engine`	debug

Build & lancement

```
cd ~/qbo_ws
colcon build --packages-select qbo_social
source install/setup.bash

ros2 launch qbo_social social.launch.py
```

Avec un fichier de paramètres custom :

```
ros2 launch qbo_social social.launch.py params_file:=/path/to/my_params.yaml
```

Debug

Dashboard temps réel (inspiré SMACH Viewer)

```
# Vue d'ensemble : modes, nœuds, batterie, pipeline intent, log événements
ros2 run qbo_social debug_behavior_state

# Mode défilement classique (sans effacement écran)
ros2 run qbo_social debug_behavior_state --scroll
```

Le dashboard affiche en temps réel :

- Mode système courant + profil actif + nœuds manquants
- État batterie (tension, runtime estimé, charge)
- Températures / RAM / puissance Orin NX
- Tableau de présence de chaque nœud hardware (vert / rouge / attente)
- WorldState (visage, personne, engagement, tracking)
- Pipeline intent : `raw → BLOQUÉ / DÉGRADÉ / OK → filtré`
- Log circulaire des 14 derniers events/intents/traces

Topics individuels

```
ros2 topic echo /qbo_social/events
ros2 topic echo /qbo_social/world_state
ros2 topic echo /qbo_social/intent
ros2 topic echo /qbo_social/decision_trace
```

Injecter un événement manuellement

```
ros2 topic pub /qbo_social/events qbo_msgs/msg/SocialEvent \
  "{event_type: 'FACE_APPEARED', source: 'test', person_id: '', person_name: '', confidence: 1.0, payload_json: '{}'}"
```

Vérifier / changer le profil actif

```
ros2 param get /qbo_system_mode_manager active_profile
ros2 param set /qbo_system_mode_manager active_profile VISION
```

Simuler une batterie faible (seuils en volts)

```
ros2 topic pub --once /diagnostics diagnostic_msgs/msg/DiagnosticArray \
  "{status: [{hardware_id: 'Qboard_3', name: 'Battery Status', level: 1, \
    message: 'Low battery', values: [{key: 'Voltage', value: '12.0'}, {key: 'Charge Mode', value: '4'}]}]}"
```

Ajouter un nouveau comportement

Exemple : le robot expire verbalement s'il n'a vu personne depuis 5 minutes.

1. `FaceAdapter` — rien à changer, `FACE_LOST` existe déjà.
2. `BehaviorEngine` — ajouter dans `_update_tracking_behavior()` :

```
if self.tracking_mode == "IDLE":
    elapsed_idle = now - (self.face_lost_since or now)
    if elapsed_idle > 300 and not self.boredom_spoken:
        self.boredom_spoken = True
        self._publish_intent(intent_type="SPEAK",
                             payload={"text": "Ça fait un moment que je suis seul…"},
                             reason="boredom")
```

- `ActionExecutor` — le `SPEAK` avec `text` direct est déjà géré.
- `SYSTEM_RULES` — aucune règle nécessaire sauf si le comportement doit être bloqué en mode `CHARGING` par exemple.

Exemple de sortie du dashboard

```
╔═══════════════════════════════════════════════════════════════════════════════════════╗
║                              QBO SOCIAL BEHAVIOR ENGINE — DASHBOARD                   ╒
╟═══════════════════════════════╤═══════════════════════════════════════════════════════╢
║  MODE   : NOMINAL             ║  NŒUDS HARDWARE                                       ╒
║  PROFIL: VISION               ║  System         ✓   Qboard_1        ✓   Qboard_3        ✓   ╒
║  BATT   : 12.6V ▓▓▓
```

Colonne	Description	Exemple
Notes / À valider	Questions, seuils à confirmer sur robot	vérifier sur Orin réel

Règles existantes (référence)

ID	Situation	Source hw	name diag	Clé values	Cible robot_state	Condition	Mode	Intents bloqués
`CRITICAL_BATTERY`	Tension critique	`Qboard_3`	`Battery Status`	`Voltage`	`battery_voltage`	`< 11.5 V`	`CRITICAL`	tous
`LOW_BATTERY`	Tension faible	`Qboard_3`	`Battery Status`	`Voltage`	`battery_voltage`	`< 12.2 V`	`LOW_BATTERY`	—
`CHARGING`	En charge	`Qboard_3`	`Battery Status`	`Charge Mode`	`charging`	`1 ou 2`	`CHARGING`	—
`CPU_OVERLOAD`	Orin trop chaud	`orin-nx-16g`	`A608 Temp`	`CPU °C`	`cpu_temp_c`	`>= 70.0`	`THERMAL_LIMIT`	—
`VISION_MISSING`	qbo_vision absent	—	—	—	`nodes_present["qbo_vision"]`	`is False`	`DEGRADED_VISION`	`TRACK_FACE_FULL` `TRACK_FACE_HEAD_ON` `START_PERSON_SEARCH`

Nouvelles règles à définir

Remplir à la main, puis transcrire dans `SYSTEM_RULES` et `DIAGNOSTIC_EXTRACTORS`.

ID	Situation	Source hw	name diag	Clé values	Cible robot_state	Condition	Mode	Intents bloqués	Dégradations	Priorité	Notes

Processus de rédaction d'une règle (étapes)

```
┌──────────────────────────────────────────────────────────────────────┐
│ 1. OBSERVER  — ros2 topic echo /diagnostics | grep <hardware_id>     │
│              → identifier hardware_id, name, clé, plage de valeurs   │
│                                                                      │
│ 2. NOMMER    — définir l'ID de règle + décrire la situation en une phrase │
│                                                                      │
│ 3. EXTRAIRE  — si la valeur n'est pas encore dans robot_state :      │
│              → ajouter une entrée dans DIAGNOSTIC_EXTRACTORS         │
│              → ajouter la valeur initiale dans robot_state           │
│                                                                      │
│ 4. CONDITION — écrire lambda s: s.get("cible", défaut) <op> seuil    │
│                                                                      │
│ 5. EFFET     — choisir : mode système, intents bloqués, dégradations │
│                                                                      │
│ 6. PRIORITÉ  — 100 = sécurité critique / 50-80 = matériel / 10-40 = soft │
│                                                                      │
│ 7. TESTER    — simuler via ros2 topic pub sur /diagnostics           │
│              → vérifier mode + dashboard + events                    │
└──────────────────────────────────────────────────────────────────────┘
```